# Unit 4: Modern Approach to Software Project

B.E. Software VIII Semester

Gandaki Engineering College of Science

Lamachaur, Pokhara

# Modern Software Development Project

- In short, software development is the overall process involved when taking a software project from beginning to production delivery.
- This process incorporates the design, documentation, programming, testing and ongoing maintenance of a software deliverable.

# Modern Software Project Management

- Modern S/W Project Management that makes this a perfect time to reflect on what modern Project Management has become and why it is such a critical issue for business organizations of all types.
- Project Management is a discipline for planning, leading, organizing and controlling a well-defined collection of work.

# Modern Software Project Management

- Software is said to be an intangible product.
- Software development is a kind of all new stream in world business and there's very little experience in building software products.
- Most software products are adapt/modify made to fit client's requirements.

# Modern Software Project Management

- The most important is that the underlying technology changes and advances so frequently and rapidly that experience of one product may not be applied to the other one.
- All such business and environmental constraints bring risk in software development hence it is essential to manage software projects efficiently.

# Modern Software Project Management


- The image above shows triple constraints for software projects.
- It is an essential part of software organization to deliver quality product, keeping the cost within client's budget constrain and deliver the project as per scheduled.

- There are several factors, both internal and external, which may impact this triple constrain triangle.
- Therefore, software project management is essential to incorporate user requirements along with budget and time constraints.

# Five Essential Elements for Successful Software Development

- Integrated Development Environment (IDE).
- Source Control.
- Automated Testing.
- Automated Build.
- Defect Management.
- **The 5 Most Important Elements of Successful Project Management**
- Have Clear Project Goals. Make sure you have all the details in front of you before you start.
- Be Dynamic. Once you have your plan in place, remain flexible.
- Communication. You need to ensure your team clearly communicates with one another.
- Stay on Track.
- Review The Project to Improve for The Next Time

# Management Activities

- Planning the works
- Measure the Project
- Manage the Risk
- Resource Planning
- Reuse
- Ensure Quality
- Communicate with stakeholders
- Manage the Project Team

# Top 10 Management Principles of Iterative Development

- Base the process on an **architecture-first approach.**
- Establish an **iterative lifecycle process that** confronts risk early.
- Transition design methods to emphasize **component based development.**
- Establish a **change-management environment.**
- Enhance **change freedom through tools.**
- Capture design artifacts in rigorous, **model-based notation.**

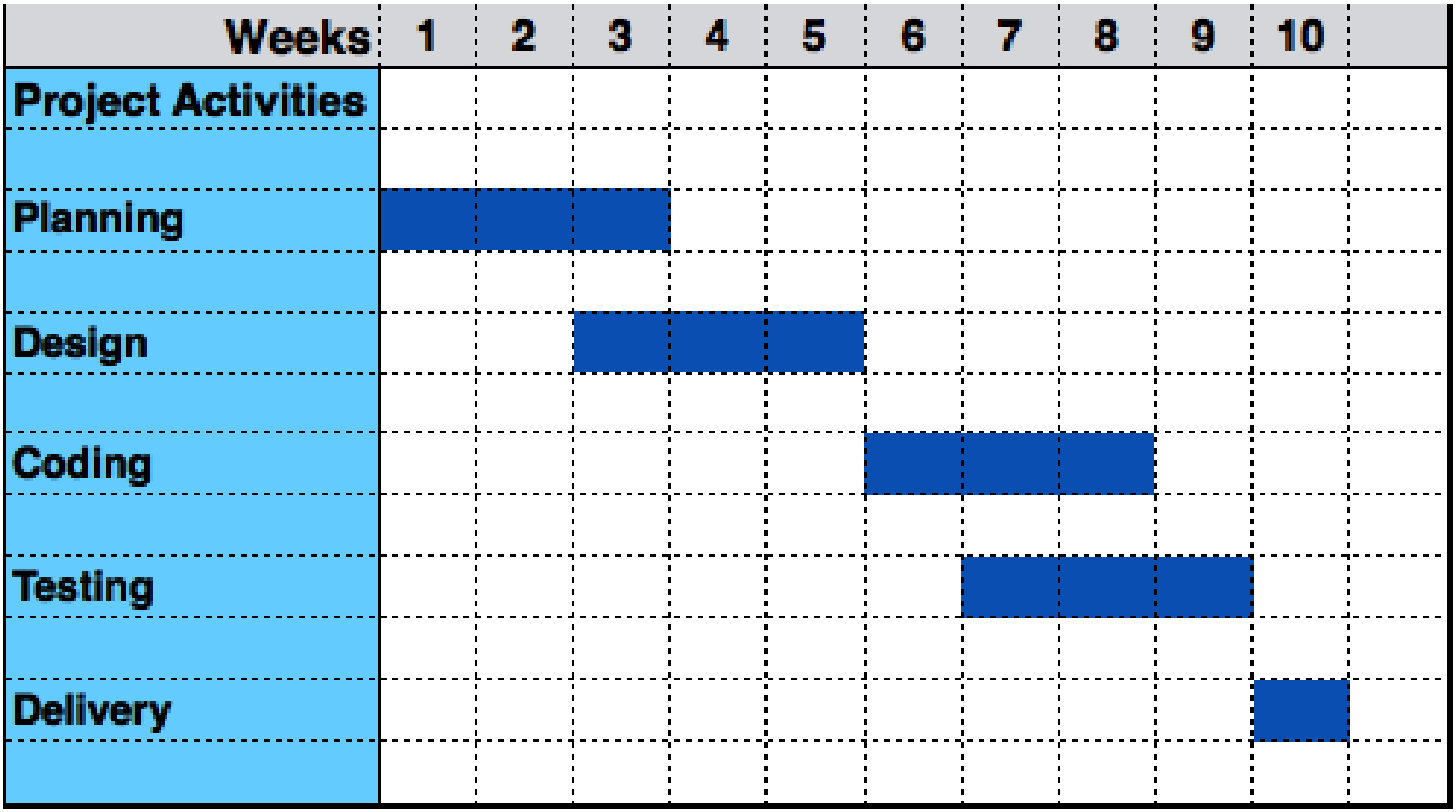
# Top 10 Management Principles of Iterative Development

- Instrument the process for **objective quality control and progress assessment.**
- Use a **demonstration-based approach to assess** intermediate artifacts.
- Plan intermediate releases in groups of usage scenarios with **evolving levels of detail.**
- Establish a **configurable process that is** economically scalable.

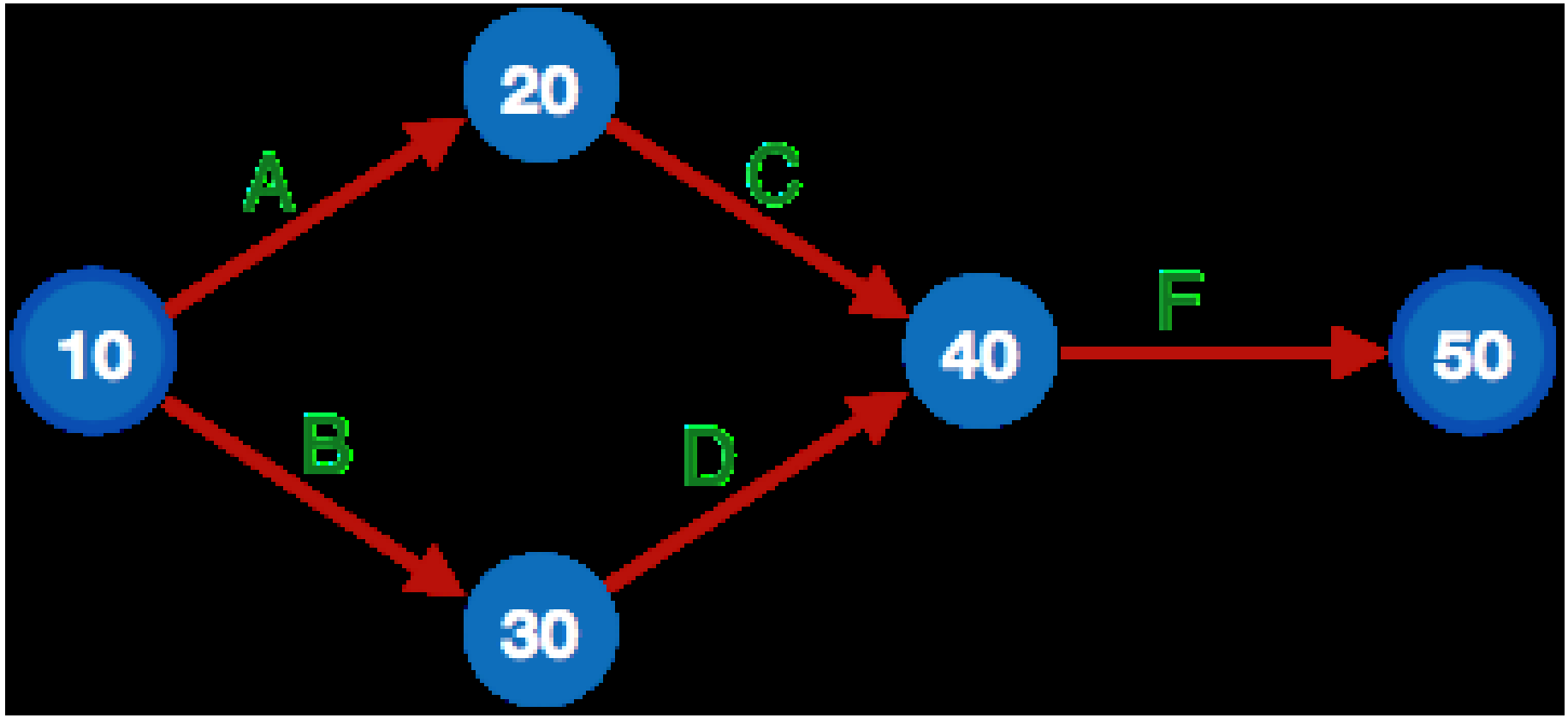
**Project management principles are necessary assets when charting a path to completion.**

- Project structure
- Definition phase
- Clear goals
- Transparency about project status
- Risk recognition
- Managing project disturbances
- Responsibility of the Project Manager
- Project success

# Project Management Tools



# PERT Chart



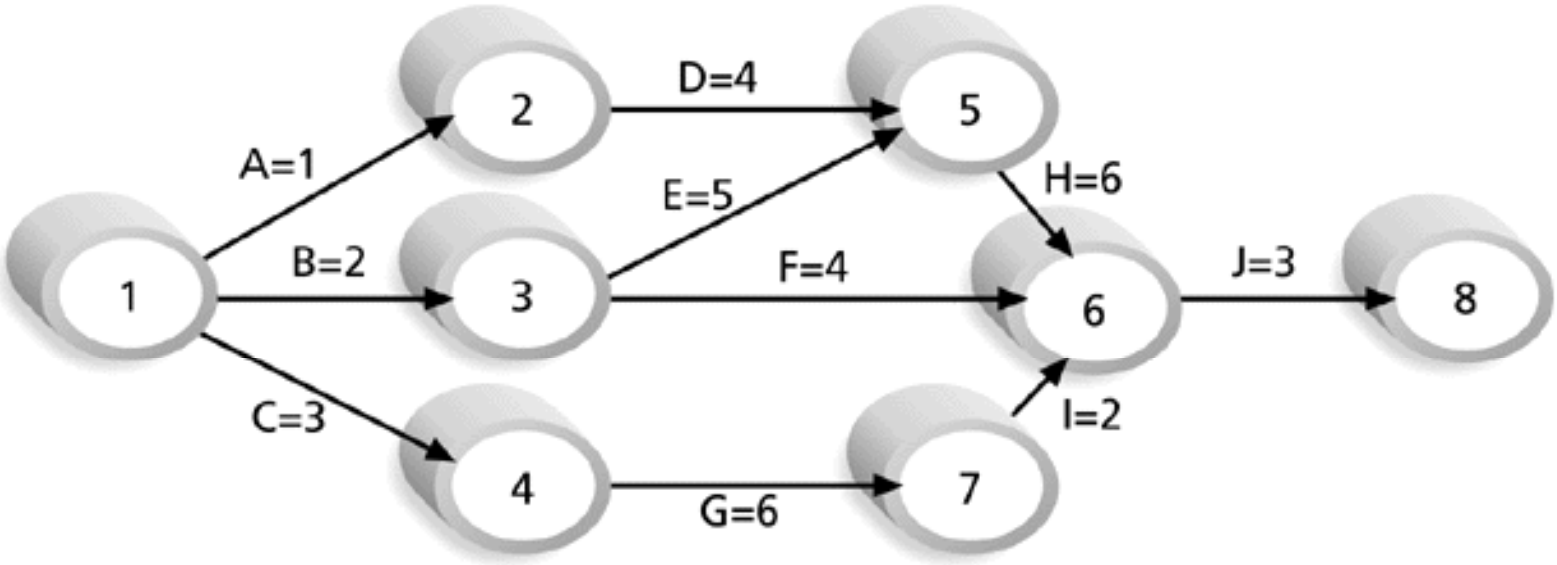
# Critical Path Method (CPM)

- **CPM** is a network diagramming technique used to predict total project duration.
- The critical path is the *longest path* through the network diagram and has the least amount of slack or float.

# Calculating the Critical Path

- Develop a good network diagram.
- Add the duration estimates for all activities on each path through the network diagram.
- The longest path is the critical path.
- If one or more of the activities on the critical path takes longer than planned, the whole project schedule will slip *unless* the project manager takes corrective action.

# Figure 6-8. Determining the Critical Path for Project X




Note: Assume all durations are in days.

| | | |
|---|---|---|
| Path 1: | A-D-H-J | Length = 1+4+6+3 = 14 days |
| **Path 2:** | **B-E-H-J** | **Length = 2+5+6+3 = 16 days** |
| Path 3: | B-F-J | Length = 2+4+3 = 9 days |
| Path 4: | C-G-I-J | Length = 3+6+2+3 = 14 days |

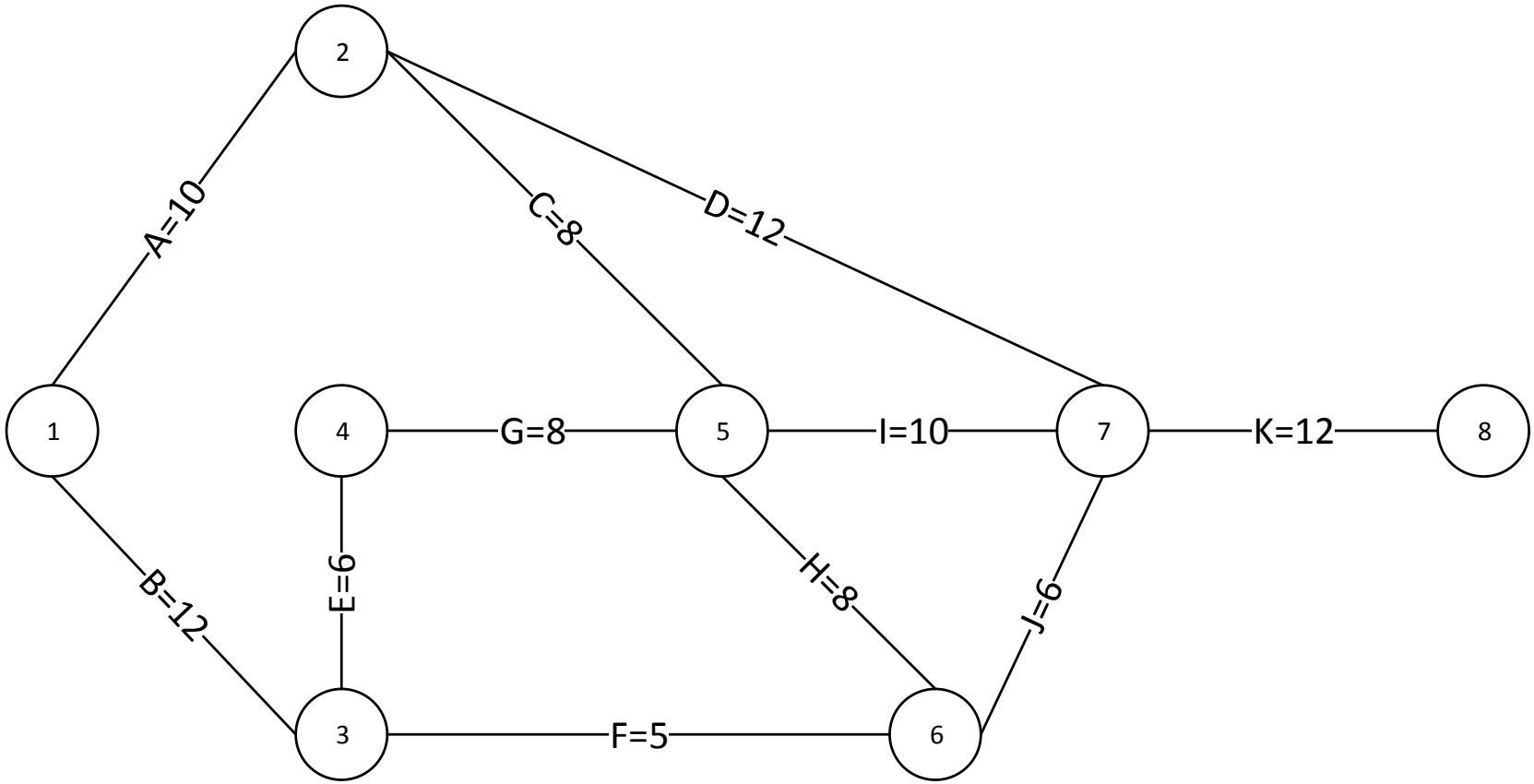
**Since the critical path is the longest path through the network diagram, Path 2, B-E-H-J, is the critical path for Project X.**

# How to Find the Critical Path

- To find the critical path, need to determine the following quantities for each activity in the network

1. *Earliest start time* (*ES*): the earliest time an activity can begin without violation of immediate predecessor requirements
2. *Earliest finish time* (*EF*): the earliest time at which an activity can end
3. *Latest start time* (*LS*): the latest time an activity can begin without delaying the entire project
4. *Latest finish time* (*LF*): the latest time an activity can end without delaying the entire project

# Critical Path Method: An Example



| Activity | D | ES (Tei) | EF | LS | LF (TLj) | TF (LS-ES) | FF=EF(succeed)-EF |
|---|---|---|---|---|---|---|---|
| 1-2 | 10 | | | | | | |
| 1-3 | 12 | | | | | | |
| 2-5 | 8 | | | | | | |
| 2-7 | 12 | | | | | | |
| 3-4 | 6 | | | | | | |
| 3-6 | 5 | | | | | | |
| 4-5 | 8 | | | | | | |
| 5-6 | 8 | | | | | | |
| 5-7 | 10 | | | | | | |
| 6-7 | 6 | | | | | | |
| 7-8 | 12 | | | | | | |

| Activity (i-j) | D | ES (Tei) | EF | LS | LF (TLj) | TF (LS-ES) | FF= ES(succeed)-EF |
|---|---|---|---|---|---|---|---|
| 1-2 | 10 | 0 | | | 10 | | |
| 1-3 | 12 | 0 | | | 12 | | |
| 2-5 | 8 | 10 | | | 26 | | |
| 2-7 | 12 | 10 | | | 40 | | |
| 3-4 | 6 | 12 | | | 18 | | |
| 3-6 | 5 | 12 | | | 34 | | |
| 4-5 | 8 | 18 | | | 26 | | |
| 5-6 | 8 | 26 | | | 34 | | |
| 5-7 | 10 | 26 | | | 40 | | |
| 6-7 | 6 | 34 | | | 40 | | |
| 7-8 | 12 | 40 | | | 52 | | |

| Activity | D tij | ES (Tei) | EF Tei+D | LS Tlj-tij | LF (TLj) | TF (LS-ES) | FF=ES(succeed)-EF |
|---|---|---|---|---|---|---|---|
| 1-2 | 10 | 0 | 10 | 8 | 18 | | |
| 1-3 | 12 | 0 | 12 | 0 | 12 | | |
| 2-5 | 8 | 10 | 18 | 18 | 26 | | |
| 2-7 | 12 | 10 | 22 | 28 | 40 | | |
| 3-4 | 6 | 12 | 18 | 12 | 18 | | |
| 3-6 | 5 | 12 | 17 | 29 | 34 | | |
| 4-5 | 8 | 18 | 26 | 18 | 26 | | |
| 5-6 | 8 | 26 | 34 | 26 | 34 | | |
| 5-7 | 10 | 26 | 36 | 30 | 40 | | |
| 6-7 | 6 | 34 | 40 | 34 | 40 | | |
| 7-8 | 12 | 40 | 52 | 40 | 52 | | |

| Activity | D tij | ES (Tei) | EF Tei+D | LS Tlj-tij | LF (TLj) | TF (LS-ES) | FF=ES(succeed)-EF |
|---|---|---|---|---|---|---|---|
| 1-2 | 10 | 0 | 10 | 8 | 18 | 8 | |
| 1-3 | 12 | 0 | 12 | 0 | 12 | 0 | |
| 2-5 | 8 | 10 | 18 | 18 | 26 | 8 | |
| 2-7 | 12 | 10 | 22 | 28 | 40 | 18 | |
| 3-4 | 6 | 12 | 18 | 12 | 18 | 0 | |
| 3-6 | 5 | 12 | 17 | 29 | 34 | 17 | |
| 4-5 | 8 | 18 | 26 | 18 | 26 | 0 | |
| 5-6 | 8 | 26 | 34 | 26 | 34 | 0 | |
| 5-7 | 10 | 26 | 36 | 30 | 40 | 4 | |
| 6-7 | 6 | 34 | 40 | 34 | 40 | 0 | |
| 7-8 | 12 | 40 | 52 | 40 | 52 | 0 | |

| Activity | D tij | ES (Tei) | EF Tei+D | LS Tlj-tij | LF (TLj) | TF (LS-ES) | FF=ES(succeed)-EF |
|---|---|---|---|---|---|---|---|
| 1-2 | 10 | 0 | 10 | 8 | 18 | 8 | 0 |
| 1-3 | 12 | 0 | 12 | 0 | 12 | 0 | 0 |
| 2-5 | 8 | 10 | 18 | 18 | 26 | 8 | 8 |
| 2-7 | 12 | 10 | 22 | 28 | 40 | 18 | 18 |
| 3-4 | 6 | 12 | 18 | 12 | 18 | 0 | 0 |
| 3-6 | 5 | 12 | 17 | 29 | 34 | 17 | 17 |
| 4-5 | 8 | 18 | 26 | 18 | 26 | 0 | 0 |
| 5-6 | 8 | 26 | 34 | 26 | 34 | 0 | 0 |
| 5-7 | 10 | 26 | 36 | 30 | 40 | 4 | 4 |
| 6-7 | 6 | 34 | 40 | 34 | 40 | 0 | 0 |
| 7-8 | 12 | 40 | 52 | 40 | 52 | 0 | 0 |

# Program Evaluation and Review Technique (PERT)

- **PERT** is a network analysis technique used to estimate project duration when there is a high degree of uncertainty about the individual activity duration estimates
- PERT uses **probabilistic time estimates**
  - duration estimates based on using optimistic, most likely, and pessimistic estimates of activity durations, or a three-point estimate

# PERT Formula and Example

- PERT weighted average **=**

$$\frac{\text{optimistic time} + 4\text{X most likely time} + \text{pessimistic time}}{6}$$

- Example:

PERT weighted average =

$$\frac{8 \text{ workdays} + 4 \text{ X } 10 \text{ workdays} + 24 \text{ workdays}}{6} = \textbf{12 days}$$

where optimistic time= 8 days

most likely time = **10 days**, and

pessimistic time = 24 days

Therefore, you'd use **12 days** on the network diagram instead of 10 when using PERT for the above example

# Program Evaluation Review Technique (PERT)

- The **optimistic** time estimate (Ote)
- The **pessimistic** time estimate (Pte)
- The **most likely** time estimate (Mlte)
- **Expected time** (Te)

# Example

**tell me:**

**(a) if every thing progress as you expect, say in complete ideal situation how long it will take?**

**(b) if every thing goes wrong and encountered unexpected situations, say in worst condition how long it will take? And**

**(c) in carrying out such work what would be the normal duration?**

# Expected time

$$Te = \frac{Ote + 4Mlte + Pte}{6}$$

Let’s say:

- in first case, the time required would be to weeks
- in second case 34 weeks, and
- in third 16 weeks.

# The Expected Time

$$te = \frac{to + (4 \times 16) + 34}{6}$$

$$= 27 \text{ weeks}$$

# Agile and Time Management

- Core values of the Manifesto for Agile Software Development are
  - Customer collaboration over contract negotiation
  - Responding to change over following a plan
- The product owner defines and prioritizes the work to be done within a spring, so collaboration and time management are designed into the process
- Teams focus on producing a useful product in a specified timeframe with strong customer input
- Don’t emphasize defining all the work before scheduling it

# Schedule Control Suggestions

- Perform reality checks on schedules
- Allow for contingencies
- Don’t plan for everyone to work at 100% capacity all the time
- Hold progress meetings with stakeholders and be clear and honest in communicating schedule issues

# Controlling the Schedule

- Goals are to know the status of the schedule, influence factors that cause schedule changes, determine that the schedule has changed, and manage changes when they occur
- Tools and techniques include
  - Progress reports
  - A schedule change control system
  - Project management software, including schedule comparison charts like the tracking Gantt chart
  - Variance analysis, such as analyzing float or slack
  - Performance management, such as earned value

# Reality Checks on Scheduling

- First review the draft schedule or estimated completion date in the project charter
- Prepare a more detailed schedule with the project team
- Make sure the schedule is realistic and followed
- Alert top management well in advance if there are schedule problems

# Working with People Issues

- Strong leadership helps projects succeed more than good PERT charts
- Project managers should use
  - empowerment
  - incentives
  - discipline
  - negotiation

# Using Software to Assist in Time Management

- Software for facilitating communications helps people exchange schedule-related information
- Decision support models help analyze trade-offs that can be made
- Project management software can help in various time management areas

# Software Management Best Practices

- **There is nine best practices:**
  - Formal risk management
  - Agreement on interfaces
  - Formal inspections
  - Metric-based scheduling and management
  - Binary quality gates at the inch- pebble level
  - Program-wide visibility of progress versus plan
  - Defect tracking against quality targets
  - Configuration management
  - People-aware management accountability

# Next-Generation Software Economics & Cost Models

- Next generation software economics is being practiced by some advanced software organizations.
- Software experts hold widely different opinions about software economics and its symptom in software cost estimation models:
  - Source lines of code versus function points.
  - Economy of scale versus diseconomy of scale.
  - Productivity measures versus quality measures.
  - Java versus C++
  - Object Oriented versus Function Oriented
  - Commercial components versus custom development

$$\text{Effort} = F(T_{Arch}, S_{Arch}, Q_{Arch}, P_{Arch}) + F(T_{App}, S_{App}, Q_{App}, P_{App})$$

$$\text{Time} = F(P_{Arch}, \text{Effort}_{Arch}) + F(P_{App}, \text{Effort}_{App})$$

where:

T = technology parameter (environment automation support)
S = scale parameter (such as use cases, function points, source lines of code)
Q = quality parameter (such as portability, reliability, performance)
P = process parameter (such as maturity, domain experience)

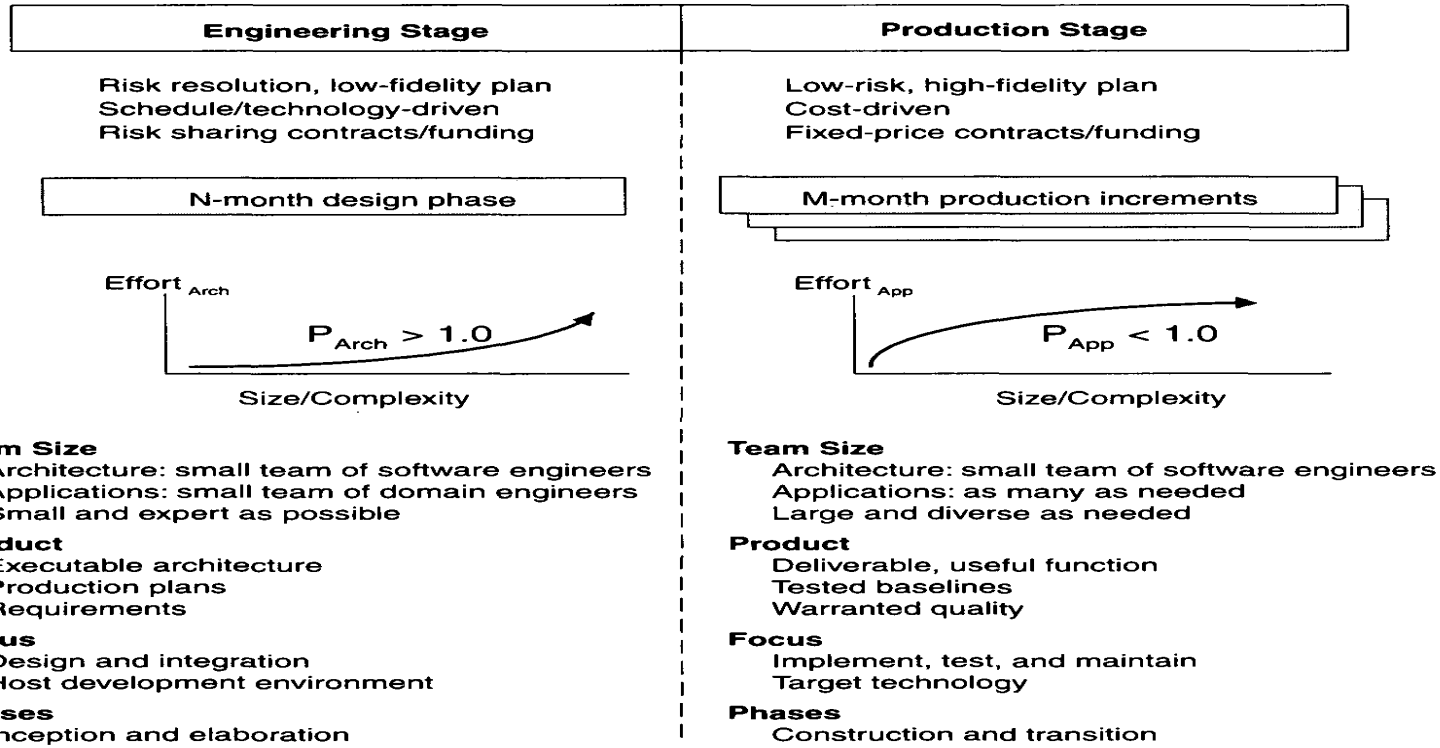



FIGURE 16-1. *Next-generation cost models*

- It will be difficult to improve based on estimation models while the project data going into these models are highly lacking a mutual relationship , and are based on differing process and technology foundations.

- Some of today's popular software cost models are not well matched to an computational software process focused an architecture-first approach
- Many cost estimators are still using a traditional or common process experience base to estimate a modern project profile
- Then, a next-generation software cost model should clearly separate architectural engineering from application production, just as an architecture-first process does.

- There are two major improvements in next generation software cost estimation models:-
- Separation of the engineering stage from the production stage: which will force estimators to differentiate between architectural scale and implementation size.
- Detailed design notations such as UML will offer an opportunity to define units of measure for scale that are more standardized and therefore can be automated and tracked.

# Modern Software Economics

- Changes: that provide a good description of what an organizational manager should attempt for in making the transition to a modern process:
- Finding and fixing a software problem after delivery costs is 100 times more than fixing the problem in early design phases.
- We can compress software development schedules 25% of nominal, but no more.
- For every $100 you spend on development, you will spend $200 on maintenance.
- Software development and maintenance costs are primarily a function of the number of source lines of code.

# Modern Software Economics

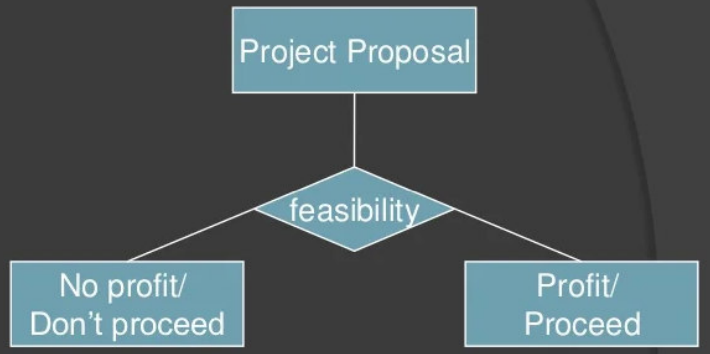
- Variations among people account for the biggest differences in software productivity.
- Only about 15% of software development effort is devoted to programming.
- Software systems and products cost 3 times as much per SLOC as individual software programs.
- 8.80% of the contribution comes from 20% of the contributors.

# Software Economics

- Software economics is situated at intersection of information economics and software design and engineering.
- The goal is to understand the relationships between economic objectives, constraints, and conditions and technical software issues.
- Then use this understanding to improve software productivity.

# IMPORTANCE

- Feasibility analysis.




- ROI (Return Over Investment).

3 of 12

# Software Economics

- Five fundamental parameters that can be abstracted from software costing models:
  - Size
  - Process
  - Personnel
  - Environment
  - Required Quality

# Size

- Usually measured in SLOC or number of Function Points required.
  - SLOC (Source Line of Code) – a better metric later in project.
    - Software metric used to measure the amount of code in a software program.
  - Function Points – a better metric earlier in project.
    - Objective and structured technique to measure software size by quantifying its functionality provided to the user, based on the requirements and logical design.
    - Breaks the system into smaller components so they can be better understand and analyzed.

# Process

- Methods and techniques use to achieve goals i.e. software product.
- Process – used to guide all activities.
  - Workers (roles), artifacts, activities…
  - Support heading toward target and eliminate non-essential / less important activities
  - Process **critical** in determining software economics
    - Component-based development; application domain…iterative approach, use-case driven…
  - Movement toward '**lean**' … everything!

# Personnel

- People factors
- Personnel – capabilities of the personnel in general and in the application domain in particular
  - Motherhood:  get the right people;  good people; Can't always do this.
  - Much specialization nowadays.  Some are terribly expensive.
  - Emphasize 'team' and team responsibilities…Ability to work in a team;
    - Several newer light-weight methodologies are totally built around a team or very small group of individuals…

# Environment

- The tools / techniques / automated procedures/Software & Hardware used to support the development effort.
  - Integrated tools; automated tools for modeling, testing, configuration, managing change, defect tracking, etc…

# Required Quality

- The functionality provided performance, reliability, maintainability, scalability, portability, user interface utility; usability…

# Parameter Relationship

## PARAMETER RELATIONSHIP

- The relationships among these parameters in modeling the estimated effort can be expressed as follows:

$$\text{Effort} = (\text{Size}^{\text{Process}}) * (\text{Personnel}) * (\text{Environment}) * (\text{Quality})$$

# Software Cost Estimation

- Set of techniques and procedures that is used to drive the software cost estimation.
- It accounts for all the items that will generally be included in the general contractor's bid.
- Break down the items of work using standard format and determining the cost of each item from experience and a database of current construction cost information.
- Cost modeling practitioners often have titles of cost estimators, cost engineers or parametric analysts.

# Why To Use Cost Estimation?

- Enables you to weigh benefits against cost to see whether the project makes sense.
- Allows you to see whether the necessary funds are available to support the project.
- Serves as a guideline to help ensure that you have sufficient funds to complete the project.

# Benefits of Cost Estimation

- Cost estimate is a valuable tool for decision making.
- Provides a starting point from which to begin evaluation of a project.
- Allows comparisons to be made between investments or projects.
- Becomes easier to exclude bad projects from consideration.

# Software Cost Estimation Techniques

- In the actual cost estimation process there are other inputs and constraints that needed to be considered besides the cost drivers.
- One of the primary constraints of the software cost estimate is the financial constraint, which are the amount of the money that can be budgeted or allocated to the project. There are other constraints such as manpower constraints, and date constraints.

# Estimation Techniques

- Algorithmic (Parametric) Model
- Expert Judgment (Expertise Based)
- Top-down
- Bottom-up
- Estimation by Analogy
- Pricing to win Estimation

# Algorithmic Cost Modeling

- Use of mathematical equations to predict cost estimations.
- Equations are based on theory or historical data.
- An algorithmic cost model can be built by analyzing the cost and attributes of completed projects and finding the closest fit formula to actual experience.

# Advantages & Disadvantages of Algorithmic Cost Modeling

- **Advantages:**
  - No detailed basis
  - Easy to modify input data
  - Easy to refine and customize formulas.
  - Objectively calibrated to experience.
- **Disadvantages:**
  - Unable to deal with exceptional conditions.
  - Some experience and factors can not be quantified.
  - Sometimes algorithms may be proprietary.

# Expert Judgment

- Several experts on the proposed software development techniques and the application domain are consulted.
- They each estimate the project cost. These estimates are compared and discussed. The estimation process iterates until an agreed estimate is reached.
- This technique captures the experience and the knowledge of the estimator who provides the estimate based on their experience from a similar project to which they have participated.

# Advantages of Expert Judgment

- Relatively cheap estimation method.
- Can be accurate if experts have direct experience of similar systems.
- Useful in the absence of quantified, empirical data.
- Can factor in differences between past project experiences and requirements of the proposed project.
- Can factor in impacts caused by new technologies, applications and languages.

# Disadvantages of Expert Judgment

- Estimate is only as good expert's opinion.
- It is very inaccurate if there are no experts!
- Hard to document the factors used by the experts.

# Pricing to Win

- The software cost is estimated to be whatever the customer has available to spent on the project.
- The estimated effort depends on the customer's budget and not on the software functionality.
- In other words the cost estimate is the price that is necessary to win the contract or the project.

# Advantages & Disadvantages of Pricing to Win

- **Advantages:**
  - Often rewarded with the contract.
- **Disadvantages:**
  - Time and money run out before the job is done.
  - The probability that the customer gets the system he or she wants is small. Costs do not accurately reflect the work required.

# Estimation by Analogy

- This technique is applicable when other projects in the same application domain have been completed. The cost of new project is estimated by analogy with these completed projects.
- Comparing the proposed project to previous completed similar project in the same application domain.
- The actual data from the completed projects are extrapolated. Can be used either at system or component level.

# Advantages & Disadvantages of Estimation by Analogy

- **Advantages:**
  - Based on actual project data.
  - It is accurate if the project data available.
- **Disadvantages:**
  - Impossible if no comparable project had been tackled in the past.
  - How well does the previous project represent this one.

# Bottom Up

- Cost of each software component is estimated and then combine the results to arrive the total cost for the project.
- The goal is to construct the estimate of the system from the knowledge accumulated about the small software components and their interactions.

# Advantages & Disadvantages of Bottom Up

- **Advantages:**
  - More stable
  - More detailed
  - Allow each software group to hand an estimate
- **Disadvantages:**
  - May overlook system level costs
  - More time consuming

# Top-Down

- This technique is also called macro model, which utilize the global view of the product and then partitioned into various low level components.
- Start at the system level and assess the overall system functionality and how this is delivered through sub systems.

# Advantages & Disadvantages of Top-Down

- **Advantages:**
  - Requires minimal project detail.
  - Usually faster and easier to implement.
  - Focus on system level activities.
- **Disadvantages:**
  - Tend to overlook low level components.
  - No detailed basis.

# Notice the Process Trends….for three generations of software economics

- Conventional development (60s and 70s)
  - Application – custom;  Size – 100% custom
  - Process – ad hoc …(discuss) – laissez faire;
  - 70s - SDLC;  customization of process to domain / mission, structured analysis, structured design, code.
- Transition (80s and 90s)
  - Environmental/tools – some off the shelf.
    - Tools:  separate, that is, often not integrated esp. in 70s…
  - Size:  30% component-based;  70% custom
  - ➔ Process:  repeatable
- Modern Practices (2000 and later)
  - Environment/tools:  off-the-shelf;  integrated
  - Size:  70% component-based;  30% custom
  - Process:  managed; measured  (refer to CMM)

# Notice the Process Trends….for three generations of software economics

- Conventional:  Predictably bad: (60s/70s)
  - usually always over budget and schedule;  missed requirements
    - All custom components;  symbolic languages (assembler); some third generation languages (COBOL, Fortran, PL/1)
    - Performance, quality almost always less than great.
- Transition:  Unpredictable (80s/90s)
    - Infrequently on budget or on schedule
    - Enter software engineering;  'repeatable process;'  project management
    - Some commercial products available – databases, networking, GUIs; But with huge growth in complexity, (especially to distributed systems) existing languages and technologies not enough for desired business performance
- Modern Practices:  Predictable (>2000s)
    - Usually on budget;  on schedule.  Managed, measured process management.  Integrated environments;  70% off-the-shelf components.  Component-based applications RAD;  iterative development;  stakeholder emphasis.

# All Advances Interrelated…

- Improved ‘process’ requires ‘improved tools’ (environmental support…)
- Better ‘economies of scale’ because
  - ➔ Applications live for years;
  - Similarly-developed applications – common.
  - First efforts in common architectures, processes, iterative processes, etc., all have **initial** high overhead;
  - But follow-on efforts result in economies of scale…and much better ROI.
  - “All simple systems have been developed!”

# “Pragmatic” Software Cost Estimation

- Little available on estimating cost for projects using iterative development.
  - Difficult to hold all the controls constant
    - Application domain; project size; criticality; etc. Very ‘artsy.’
    - Metrics (SLOC, function points, etc.) NOT consistently applied EVEN in the same application domain!
    - Definitions of SLOC and function points are not even consistent!
  - Much of this is due to the nature of development. There is no magic date when design is ‘done;’ or magic date when testing ‘begins’ …
  - Consider some of the issues:

# Three Issues in Software Cost Estimation:

- 1. Which cost estimation model should be used?
- 2. Should software size be measured using SLOC or Function Points? (there are others too…)
- 3. What are the determinants of a good estimate? (How do we know our estimate is good??)

**So very much is dependent upon estimates!!!!**

# Cost Estimation Models

- Many available.
- Many organization-specific models too based on their own histories, experiences…
  - Oftentimes, these are super if 'other' parameters held constant, such as process, tools, etc. etc.
- COCOMO, developed by Barry Boehm, is the most popular cost estimation model.
- Two primary approaches:
  - Source lines of code (SLOC) and
  - Function Points (FP)

# Source Lines of Code (SLOC)

- Many feel comfortable with 'notion' of LOC
- SLOC has great value – especially where applications are custom-built.
  - Easy to measure & instrument – have tools.
  - Nice when we have a history of development with applications and their existing lines of code and associated costs.

- Today – with use of components, source-code generation tools, and objects have rendered SLOC somewhat ambiguous.
  - We often don't know the SLOC – but do we care? How do we factor this in? →

# Function Points

- Function Points measure numbers of
  - external user inputs,
  - external outputs,
  - internal data groups,
  - external data interfaces,
  - external inquiries, etc.
- ➔ Major disadvantage: Difficult to measure these things.
  - Definitions are primitive and inconsistent
  - Metrics difficult to assess especially since normally done earlier in the development effort using more abstractions.
- Yet, no project will be started without estimates!!!!

# But:

- Cost estimation is a real necessity!!! Necessary to 'fund' project!
- All projects require estimation in the beginning (inception) and adjustments…
  - These must stabilize; They are rechecked…
  - Must be **reusable** for additional cycles
  - Can create organization's own methods of measurement on how to 'count' these metrics...

- No project is arbitrarily started without cost / schedule / budget / manpower / resource estimates (among other things)
- ➔ SO critical to budgets, resource allocation, and to a host of stakeholders

# So, How Good are the Models?

- COCOMO is said to be 'within 20%' of actual costs '70% of the time.' (COCOMO has been revised over the years…)

- Cost estimating is still **disconcerting** when one realizes that there are already a plethora of missed dates, poor deliverables, and significant cost overruns that characterize traditional development.

- Yet, all non-trivial software development efforts require costing; It is a basic management activity.

- RFPs on contracts force contractors to estimate the project costs for their survival.
- So, let's look at top down and bottom up estimating.

# Top Down versus Bottom Up Substantiating the Cost…

- Most estimators perform bottom up costing - **substantiating** a target cost - rather than approaching it a top down, which would yield a 'should cost.'
- Many project managers create a 'target cost' and then play with parameters and sizing until the target cost can be justified…
  - Work backwards!
  - Attempts to win proposals, convince people, …
- Any approach should force the project manager to assess risk and discuss things with stakeholders…

# A Good Project Estimate:

- ➔Is conceived and supported by the project manager, architecture team, development team, and test team accountable for performing the work.
- ➔Is accepted by all stakeholders as ambitious but doable
- Is based on a well-defined software cost model with a credible basis
- ➔ Is based on a database of relevant project experience that includes similar processes, similar technologies, similar environments, similar quality requirements, and similar people, and
- ➔ Is defined in enough detail so that its key risk areas are understood and the probability of success is objectively assessed.

# Any Queries?